



AI GAP-Chatbot

Complete Project Documentation

LLaMA 3.3 70B | Groq API | ChromaDB | SQLite | Streamlit

□ Project AI RAG-Chatbot	⚡ LLM LLaMA 3.3 70B	□ Vector DB ChromaDB	□ Storage SQLite DB
-----------------------------	------------------------	-------------------------	------------------------

1. Project Overview

AI GAP-Chatbot is a Retrieval-Augmented Generation (RAG) based intelligent chatbot that allows users to upload documents and ask natural language questions. The system retrieves the most relevant content from uploaded documents and uses LLaMA 3.3 70B (via Groq API) to generate accurate, context-grounded answers at lightning speed.

Key Highlights

- Upload PDF, DOCX, TXT, MD, CSV documents and chat with them instantly
- Ultra-fast LLM inference using Groq's custom LPU hardware (~300 tokens/sec)
- Semantic search powered by sentence-transformers + ChromaDB vector database
- Persistent chat history stored in SQLite with session management
- Clean dark-themed Streamlit UI with real-time streaming responses
- Fully free to build and run — no OpenAI or credit card required

□ **What is RAG?** Retrieval-Augmented Generation (RAG) combines information retrieval with text generation. Instead of relying only on a model's training data, RAG first searches a knowledge base (your documents) for relevant context, then feeds that context to the LLM to generate a grounded, accurate answer.

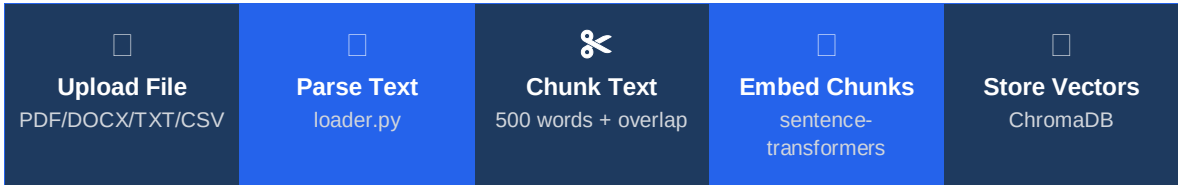
2. System Architecture

The system is divided into four major layers: Document Ingestion, Vector Storage, Retrieval Engine, and LLM Generation. Each layer is independently modular and can be swapped without affecting others.

Layer	Component	Responsibility
☐ UI Layer	Streamlit (app.py)	User interface, file upload, chat display, streaming output
☐ Ingestion Layer	loader.py + chunker.py	Parse documents, split into 500-word overlapping chunks
☐ Embedding Layer	embedder.py (sentence-transformers)	Convert text chunks and queries into dense vector embeddings
☐ Vector Store	vector_store.py (ChromaDB)	Store, index, and retrieve top-K semantically similar chunks
☐ Retrieval Layer	retriever.py	Embed user query, search ChromaDB, format retrieved context
☐ LLM Layer	groq_client.py (LLaMA 3.3 70B)	Build system prompt with context, call Groq API, stream reply
☐ Persistence Layer	helpers.py (SQLite)	Save chat history, document metadata, manage sessions

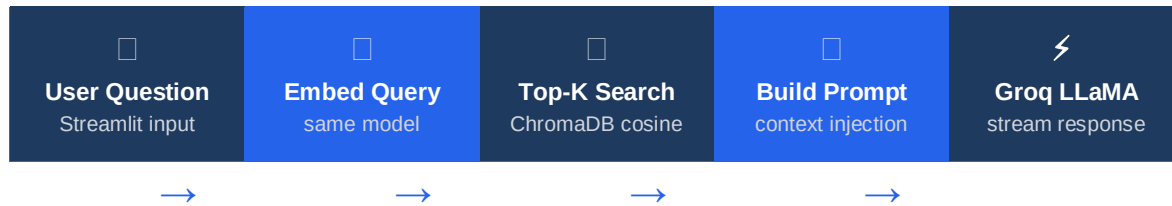
3. System Flow Diagram

Phase 1 — Document Ingestion Flow





Phase 2 — Query & Response Flow



Step-by-Step Detailed Flow

#	Step	Details
1	User uploads document	User selects PDF/DOCX/TXT/CSV via Streamlit file uploader widget
2	File saved temporarily	File written to /data/uploads/ as a temp file for processing
3	Text extraction	loader.py reads the file using pdfplumber / python-docx / pandas based on type
4	Text chunking	chunker.py splits text into 500-word chunks with 50-word overlap to preserve context
5	Embedding generation	embedder.py uses sentence-transformers all-MiniLM-L6-v2 to generate 384-dim vectors
6	ChromaDB storage	vector_store.py stores chunk text + embedding + metadata (source, doc_id, chunk index)
7	SQLite metadata save	helpers.py saves document name, file type, chunk count to SQLite documents table
8	User types question	Input captured by st.chat_input(), appended to session message history

#	Step	Details
9	Query embedding	The question is embedded with the same sentence-transformers model
10	Semantic retrieval	ChromaDB cosine similarity search returns top-4 most relevant chunks
11	Prompt construction	groq_client.py builds system prompt with retrieved chunks as context
12	LLM API call	Groq API called with LLaMA 3.3 70B model, streaming enabled (temperature 0.2)
13	Streamed response	Response tokens streamed live to Streamlit UI via st.write_stream()
14	History persistence	Full conversation turn (user + assistant) saved to SQLite chat_history table

4. Project Structure

File / Module	Purpose
app.py	Main Streamlit application — UI, routing, streaming chat
.env / .env.example	Environment variables — API keys and config
requirements.txt	All Python dependencies with pinned versions
rag/loader.py	Document parser — handles PDF, DOCX, TXT, MD, CSV
rag/chunker.py	Text splitter — 500-word chunks with 50-word overlap
rag/embedder.py	Embedding model wrapper — sentence-transformers
rag/vector_store.py	ChromaDB CRUD — add, query, delete, reset operations
rag/retriever.py	High-level retriever — embed query, fetch top-K, format context
llm/groq_client.py	Groq API client — system prompt builder, streaming LLaMA calls
utils/helpers.py	SQLite DB helper — chat history, document records, session IDs
data/uploads/	Temporary storage folder for user-uploaded files

5. Libraries Used

Core AI & ML Libraries

Library	Version	Purpose
groq	0.9.0	Official Groq Python SDK — connects to Groq cloud for ultra-fast LLaMA inference via REST API

Library	Version	Purpose
sentence-transformers	3.0.1	Generates dense vector embeddings from text using pre-trained transformer models (all-MiniLM-L6-v2)
chromadb	0.5.0	Local vector database — stores embeddings and enables cosine similarity search for RAG retrieval

Document Processing Libraries

Library	Version	Purpose
pdfplumber	0.11.0	Extracts text and tables from PDF files with high accuracy, supports complex layouts
python-docx	1.1.2	Reads and writes Microsoft Word .docx files — extracts paragraphs and document structure
pandas	2.2.2	Reads CSV files and converts tabular data to structured text for embedding

Web & Infrastructure Libraries

Library	Version	Purpose
streamlit	1.36.0	Python web framework for building interactive data apps — used for the entire chatbot UI
python-dotenv	1.0.1	Loads environment variables from .env file into Python os.environ at runtime
sqlite3	built-in	Python built-in SQLite interface — used for chat history and document metadata persistence

Library Deep Dives

□ **Groq SDK** (*pip install groq*)

□ **sentence-transformers** (*pip install sentence-transformers*)

□ **ChromaDB** (*pip install chromadb*)

6. Top Interview Questions & Answers

These are the most common questions interviewers ask about RAG systems, LLMs, and the specific libraries used in this project. Study these carefully before your interview.

A. RAG System Concepts

Q: What is RAG and why is it better than using an LLM alone?

A: RAG (Retrieval-Augmented Generation) combines a retrieval system with a language model. Instead of relying only on the model's training data, RAG first searches a knowledge base for relevant documents, then passes that context to the LLM. This reduces hallucinations, allows the model to answer from private/recent data, and keeps responses grounded and citable.

Q: What is the difference between RAG and fine-tuning?

A: Fine-tuning permanently updates the model's weights by training on new data — expensive, slow, and hard to update. RAG keeps the model frozen and retrieves fresh context at inference time — cheaper, faster to update (just add documents), and more transparent. RAG is preferred for dynamic knowledge bases; fine-tuning is better for teaching the model new behavior or style.

Q: What is chunking and why do we use overlap?

A: Chunking splits large documents into smaller pieces (e.g. 500 words) so they can be embedded and retrieved individually. Overlap (e.g. 50 words) means consecutive chunks share some text at their boundaries. This prevents important information that spans a chunk boundary from being lost — without overlap, a sentence split across two chunks might be missed during retrieval.

Q: How does semantic search differ from keyword search?

A: Keyword search (like BM25 or SQL LIKE) matches exact words. Semantic search uses vector embeddings to compare the meaning of text — so 'car' and 'automobile' would score as similar even though they share no words. In RAG, semantic search retrieves contextually relevant chunks even when the user's phrasing doesn't match the document text exactly.

Q: What is cosine similarity and why is it used for vector search?

A: Cosine similarity measures the angle between two vectors. A value of 1.0 means identical direction (highly similar), 0 means no relationship, -1 means opposite. It's preferred over Euclidean distance for text embeddings because it focuses on directional similarity rather than magnitude — two sentences can have similar meaning regardless of their word count.

B. Library-Specific Questions

Q: Why did you choose Groq instead of OpenAI?

A: Groq offers a completely free tier with no credit card required, making it ideal for student projects. More importantly, Groq's LPU (Language Processing Unit) hardware delivers ~300 tokens/second — roughly 5-10x faster than OpenAI's API. LLaMA 3.3 70B on Groq performs close to GPT-4 on most tasks while being open-source and free.

Q: What is ChromaDB and how does it store vectors?

A: ChromaDB is an open-source vector database. It stores each chunk as a document (text), an embedding (float array), metadata (source file, chunk index), and an ID. Internally it uses HNSW (Hierarchical Navigable Small World) graphs for approximate nearest neighbor search. With PersistentClient, data is saved to disk as SQLite files under the chroma_store directory.

Q: What is HNSW and why does ChromaDB use it?

A: HNSW (Hierarchical Navigable Small World) is an approximate nearest neighbor algorithm that builds a multi-layer graph of vectors. Instead of comparing the query to every stored vector (brute force $O(n)$), HNSW navigates the graph in $O(\log n)$ time. This makes vector search fast even with millions of documents, at a small cost of approximation accuracy.

Q: What embedding model did you use and why?

A: all-MiniLM-L6-v2 from sentence-transformers. It produces 384-dimensional embeddings, is only 80MB, runs on CPU without a GPU, and is very fast (~14,000 sentences/sec on CPU). Despite its small size, it scores well on semantic similarity benchmarks. It's the ideal choice for local RAG projects where cost and speed matter.

Q: Why use SQLite alongside ChromaDB?

A: ChromaDB is a vector store — it's optimized for similarity search on embeddings, not for structured relational queries. SQLite handles the relational data: chat history (who said what, when), document metadata (filename, upload time, chunk count), and session management. Using both databases together follows the principle of using the right tool for the right job.

Q: How does pdfplumber work internally?

A: pdfplumber is built on top of pdfminer.six, a pure-Python PDF parsing library. It extracts text by parsing the PDF's content streams, which contain drawing operations and text positioning instructions. It handles multi-column layouts and can extract tables as structured data. For scanned PDFs (images), it cannot extract text — OCR (like pytesseract) would be needed.

C. Design & Architecture Questions

Q: How would you scale this project to production?

A: Replace SQLite with PostgreSQL for multi-user concurrent writes. Replace local ChromaDB with a managed vector DB like Pinecone or Weaviate. Add a FastAPI backend instead of Streamlit. Containerize with Docker and deploy on Kubernetes. Add an async queue (Celery + Redis) for document processing. Add user authentication with JWT tokens.

Q: What is the context window and how does RAG work around it?

A: LLMs have a maximum context window (e.g. 128K tokens for LLaMA 3.3). Feeding entire documents would often exceed this limit and also dilute the model's attention. RAG solves this by only injecting the top-K most relevant chunks (e.g. 4 chunks of 500 words = ~2000 tokens) instead of the whole document, keeping the prompt focused and within limits.

Q: How would you evaluate the quality of your RAG system?

A: Using metrics like: Retrieval Recall (did the correct chunks get retrieved?), Answer Faithfulness (does the answer only use retrieved context?), Answer Relevance (does the answer address the question?), and Context Precision (were the retrieved chunks actually useful?). Tools like RAGAS (RAG Assessment) automate this evaluation using an LLM judge.

Q: What are the limitations of your current implementation?

A: No GPU acceleration for embeddings (CPU only, slower for large docs). Basic chunking by word count does not respect sentence or paragraph boundaries. No re-ranking of retrieved chunks. No query expansion or hypothetical document embeddings (HyDE). Single-user session model without multi-tenancy. No evaluation pipeline to measure retrieval quality.

D. Python & General Questions

Q: What is a generator and how is it used in this project?

A: A generator is a function that yields values one at a time using the yield keyword, instead of returning a complete list. In `groq_client.py`, the `chat()` function is a generator that yields each text chunk from the streaming API response. Streamlit consumes these chunks in real-time to display the response progressively, instead of waiting for the full response.

Q: What is the difference between `os.getenv()` and `dotenv`?

A: `os.getenv()` reads environment variables that are already set in the OS environment. `python-dotenv`'s `load_dotenv()` first loads variables from a `.env` file into `os.environ`, so they become available to `os.getenv()`. In production, real environment variables (set by the OS or deployment platform) take precedence over `.env` values.

Q: What is session state in Streamlit?

A: Streamlit re-runs the entire Python script from top to bottom on every user interaction. `st.session_state` is a dictionary-like object that persists data across these reruns within a browser session. In this project it stores the current `session_id` and messages list, so the chat history is not lost when the user sends a new message.

7. Quick Reference — Library Cheat Sheet

Memorize these key APIs before your interview. Interviewers love asking you to write or explain these from memory.

Operation	Code	Description
Groq — create client	<code>Groq(api_key=os.getenv('GROQ_API_KEY'))</code>	Initialize Groq client from env variable
Groq — chat call	<code>client.chat.completions.create(model=..., messages=..., stream=True)</code>	Send chat message with streaming
SentenceTransformer — load	<code>SentenceTransformer('all-MiniLM-L6-v2')</code>	Load pre-trained embedding model
SentenceTransformer — embed	<code>model.encode(texts, show_progress_bar=False)</code>	Generate embeddings for list of texts
ChromaDB — client	<code>chromadb.PersistentClient(path='./chroma_store')</code>	Create persistent disk-based client
ChromaDB — collection	<code>client.get_or_create_collection(name='rag_docs')</code>	Get or create a vector collection
ChromaDB — add	<code>collection.add(documents, embeddings, ids, metadatas)</code>	Store chunks + vectors
ChromaDB — query	<code>collection.query(query_embeddings=[q_vec], n_results=4)</code>	Find top-K similar chunks
pdfplumber — open	<code>pdfplumber.open(filepath) as pdf</code>	Open PDF file for extraction
pdfplumber — extract	<code>page.extract_text()</code>	Get plain text from a PDF page
python-docx — open	<code>docx.Document(filepath)</code>	Load a Word document
python-docx — paragraphs	<code>doc.paragraphs</code>	List of paragraph objects in the doc

Operation	Code	Description
SQLite — connect	<code>sqlite3.connect('chat_history.db')</code>	Open or create SQLite database file
SQLite — execute	<code>conn.execute(sql, params)</code>	Run a SQL query with parameters
Streamlit — chat input	<code>st.chat_input('Type here...')</code>	Render chat input box at bottom
Streamlit — session state	<code>st.session_state['key'] = value</code>	Persist data across reruns

AI RAG-Chatbot — Project Summary

LLaMA 3.3 70B • Groq API • ChromaDB • SQLite • sentence-transformers • Streamlit

Built to demonstrate Retrieval-Augmented Generation for resume and portfolio. Fully free to run. No OpenAI or paid API required.